

OpenSky Network Data & the traffic Python Library

What OpenSky actually provides, how traffic renames and exposes it, and the practical gotchas hit so far while building the Sydney congestion project.

1. What OpenSky Actually Is

OpenSky Network is a crowdsourced ADS-B/Mode S receiver network. Volunteer ground receivers around the world listen to aircraft radio broadcasts and OpenSky aggregates them into a public dataset. It does not get data from airlines, airports, or air traffic control systems — only from what aircraft themselves broadcast over the air.

Critical limitation

OpenSky has no scheduled times and no delay data. It only sees where an aircraft physically was, and when. Any “delay” or “on-time performance” figure must be derived separately — either from another data source (e.g. BITRE) or from a self-built baseline (e.g. the median duration for a given route).

Two ways to access it

- Live API — current/real-time aircraft positions. No registration required for basic use. No historical lookups.
- Trino historical interface — the full historical archive (over 40 trillion records since 2013), accessed via SQL-style queries. Requires a free account, plus separate manual approval from OpenSky administrators for historical/Trino access specifically.

2. Raw Trino Schema — state_vectors_data4

These are the actual column names inside OpenSky's database, exactly as documented and as used in raw SQL or the `selected_columns` argument when querying.

Column	Type	Description
time	int	Unix timestamp the state vector was valid at
icao24	string	Unique 24-bit transponder address (hex)
lat	double	Latitude, decimal degrees (WGS84)
lon	double	Longitude, decimal degrees (WGS84)
velocity	double	Speed over ground, metres/second
heading	double	Direction of movement, degrees from true north
vertrate	double	Vertical speed, m/s (positive = climbing)
callsign	string	Flight identifier broadcast by the aircraft
onground	boolean	True if the aircraft is on the ground
alert	boolean	ATC alert squawk active
spi	boolean	Special position indicator active
squawk	string	4-digit octal transponder code assigned by ATC
baroaltitude	double	Barometric altitude, metres
geoaltitude	double	Geometric (GNSS) altitude, metres
lastposupdate	double	Timestamp of the last recorded position update
lastcontact	double	Last time OpenSky received a signal from the aircraft
serials	array<int>	Receiver serial numbers that heard this message
hour	int	Partition column — unix timestamp of the hour this record belongs to

Performance note

hour is the partition column. Always filter on a bounded time range, or queries can return millions of rows and risk hitting OpenSky's usage limits. `flightlist()` is far lighter than `history()` for an initial scan — use it first to find candidate aircraft/flights before pulling full state vectors.

3. How the traffic Library Renames Columns

traffic does not hand back the raw schema as-is. It standardises column names to its own consistent convention across every data source it supports, so analysis code stays the same even if the underlying provider changes. This is the single most common source of KeyError when working with traffic for the first time.

Requested in selected_columns	Returned as (by traffic)
time	timestamp
lat	latitude
lon	longitude
baroaltitude	altitude
velocity	groundspeed
vertrate	vertical_rate
icao24	icao24 (unchanged)
callsign	callsign (unchanged)

Practical rule

When writing the query itself (inside `selected_columns=[...]`) use the raw schema names. When writing code that processes the result afterward (`sort_values`, filtering, plotting) use traffic's renamed columns. If a `KeyError` appears on a column name, run `print(result.data.columns.tolist())` (or `print(result.columns.tolist())` for `flightlist`, which returns a plain `DataFrame`) before assuming the query itself failed.

4. Key traffic / pyopensky Functions Used So Far

opensky.flightlist(...)

Lightweight, flight-level summary. One row per flight, not per second. The right first query for finding candidate aircraft/rotations before pulling detailed trajectories.

```
from traffic.data import opensky

flights = opensky.flightlist(
    start="2026-05-06 06:00:00",
    stop="2026-05-06 10:00:00",
    departure_airport="YSSY", # or callsign="QFA%"
)

print(flights.columns.tolist())
```

Returns a plain pandas DataFrame (no .data wrapper). Typical columns include icao24, callsign, firstseen, lastseen, estdepartureairport, estarrivalairport.

opensky.history(...)

Full, detailed state vectors — position updates roughly once per second per aircraft. Much heavier than flightlist. Use only once a specific aircraft/time window has already been identified.

```
full_day = opensky.history(
    start="2026-05-06 00:00:00",
    stop="2026-05-06 23:59:59",
    icao24="7c6c24",
    selected_columns=[
        "time", "icao24", "callsign",
        "lat", "lon", "baroaltitude",
        "velocity", "vertrate"
    ]
)

full_day = full_day.sort_values("timestamp") # renamed column, not "time"
print(full_day.data.columns.tolist())
```

Returns a traffic-specific object (Traffic/Flight class). The underlying pandas DataFrame is accessed via .data — e.g. full_day.data[["timestamp", "latitude"]].

Callsign filtering

Airlines are filtered by ICAO airline designator, not by name. Qantas mainline broadcasts as QFA followed by a flight number (e.g. QFA482) — not the word “Qantas” anywhere in the data.

Brand	ICAO designator	Notes
Qantas mainline	QFA	Use callsign="QFA%" (SQL wildcard, matches under the hood)
QantasLink	QJE	Some QantasLink flights are operated by contracted carriers under their own designators
Jetstar	JST	Separate Qantas Group airline, separate brand and callsign

5. Practical Gotchas Encountered So Far

1. Trino historical access is separate from a basic OpenSky account, and is granted manually by administrators — expect a short wait and a possible follow-up question about intended use (mention academic purpose, unit/university, and that no delay data is being requested directly).
2. Authentication uses an OAuth browser flow (`--external-authentication`), not a stored username/password file. This is the safer option — deliberately avoid creating a plaintext credentials file.
3. Unbounded queries (no tight time window, no aircraft/airport filter) can return millions of rows and take minutes. Always filter on a narrow time range first, especially while testing.
4. `pandas/traffic/pyopensky` version compatibility is fragile — a major `pandas` release (3.x) breaking `traffic`'s internals was the root cause of repeated import errors during environment setup. Once a working combination is found, freeze it (`pip freeze > requirements.txt`) and avoid further installs in that environment unless necessary.
5. Column names differ between the query stage (raw schema) and the result stage (`traffic`'s renamed schema) — see Section 3. This is the most common source of `KeyError`.

6. What Still Needs a Different Source

- Scheduled departure/arrival times — not available from OpenSky at all. Plan: derive an empirical baseline (median actual duration/turnaround per route) directly from OpenSky data itself, rather than sourcing an external timetable.
- Delay figures — not available from OpenSky. BITRE provides aggregate, route-level on-time performance statistics as a complementary public source.
- Weather — not available from OpenSky. Bureau of Meteorology (BOM) / METAR aviation reports, planned for the Question 3 stretch goal.